

Join the Vevox session

Go to **vevox.app**

Enter the session ID: **104-345-753**

Or scan the QR code





JIT stands for

Java Interpreter Translation

0%

Just In Time

0%

Juice Is Toxic

0%



50

Join at: vevox.app

ID: 104-345-753

Preparing Results



JIT stands for

Java Interpreter Translation



8%

Just In Time



92%

Juice Is Toxic



0%

RESULTS SLIDE



The CPU and GPU

splits the CPU ram into disjoint parts

0%

share the same RAM

0%

have different RAM

0%



55

Join at: vevox.app

ID: 104-345-753

Preparing Results



The CPU and GPU

splits the CPU ram into disjoint parts



0%

share the same RAM



13%

have different RAM



87%

RESULTS SLIDE



The CPU and GPU

run simultaneously and independently

0%

runs alternately (CPU -> GPU -> CPU -> ...)

0%

runs simultaneously but in sync

0%



64

Join at: vevox.app

ID: 104-345-753

Preparing Results



The CPU and GPU

run simultaneously and independently



89%

runs alternately (CPU -> GPU -> CPU -> ...)



0%

runs simultaneously but in sync



11%

RESULTS SLIDE



What is a CUDA kernel?

A function we can call from the CPU which runs on the GPU

0%

A function we can call on from GPU code

0%

A way to synchronize data to/from the GPU

0%



62

Join at: vevox.app

ID: 104-345-753

Preparing Results



What is a CUDA kernel?

A function we can call from the CPU which runs on the GPU



67%

A function we can call on from GPU code

17%

A way to synchronize data to/from the GPU

16%

RESULTS SLIDE



Let A be an array of length 100. Given 20 threads per block, how many thread blocks are needed to cover A ?

4

0%

5

0%

6

0%



59

Join at: vevox.app

ID: 104-345-753

Preparing Results

Let A be an array of length 100. Given 20 threads per block, how many thread blocks are needed to cover A?

4

0%

5

100%

6

0%

RESULTS SLIDE



Let A be an array of length 100. Given 16 threads per block, how many thread blocks are needed to cover A ?

5

0%

6

0%

7

0%



61

Join at: vevox.app

ID: 104-345-753

Preparing Results

Let A be an array of length 100. Given 16 threads per block, how many thread blocks are needed to cover A?

5



5%

6



7%

7



89%

RESULTS SLIDE



Assume x and y have length 100. With 10 threads per block, how many blocks are needed to cover y for this code?

5

0%

10

0%

100

0%

```
from numba import cuda

@cuda.jit
def double_kernel(x, y):
    i = cuda.grid(1)
    y[2*i] = 2 * x[i]
    y[2*i + 1] = 2 * x[i]
```



59

Join at: vevox.app

ID: 104-345-753

Preparing Results

Assume x and y have length 100. With 10 threads per block, how many blocks are needed to cover y for this code?

5



20%

10



63%

100



17%

```
from numba import cuda

@cuda.jit
def double_kernel(x, y):
    i = cuda.grid(1)
    y[2*i] = 2 * x[i]
    y[2*i + 1] = 2 * x[i]
```

RES

LIDE

Back to the slides

Join the Vevox session

Go to **vevox.app**

Enter the session ID: **104-345-753**

Or scan the QR code





On the GPU, operations are performed

individually for each thread

0%

in groups of 32 threads

0%

in groups corresponding to the thread block

0%



52

Join at: vevox.app

ID: 104-345-753

Preparing Results



On the GPU, operations are performed

individually for each thread



0%

in groups of 32 threads



77%

in groups corresponding to the thread block



23%

RESULTS SLIDE



If threads in a warp must perform different operations, then

the operation is skipped

0%

the GPU performs a partial warp

0%

multiple warps are performed

0%



If threads in a warp must perform different operations, then

the operation is skipped



5%

the GPU performs a partial warp



21%

multiple warps are performed



74%

RESULTS SLIDE



Let x and y be $N \times M$ matrices stored row-wise. What thread block configuration is likely to be faster?

(1, 1024)

0%

(1024, 1)

0%

```
from numba import cuda

@cuda.jit
def double_kernel_2d(x, y):
    i, j = cuda.grid(2)
    y[i, j] = 2 * x[i, j]
```



Let x and y be $N \times M$ matrices stored row-wise. What thread block configuration is likely to be faster?

(1, 1024)



36%

(1024, 1)



64%

```
from numba import cuda

@cuda.jit
def double_kernel_2d(x, y):
    i, j = cuda.grid(2)
    y[i, j] = 2 * x[i, j]
```

RESULTS SLIDE



For what values of n do we expect the best performance?

Any n

0%

Any n above the warp size

0%

Any n divisible by the warp size

0%

```
@cuda.jit
def condscale(x, y, n):
    i = cuda.grid(1)
    for _ in range(100):
        if (i // n) % 2 == 0:
            y[i] = x[i] * 3
        else:
            y[i] = x[i] / 3
```



56

Join at: vevox.app

ID: 104-345-753

Preparing Results



For what values of n do we expect the best performance?

Any n



2%

Any n above the warp size



9%

Any n divisible by the warp size



89%

```
@cuda.jit
def condscale(x, y, n):
    i = cuda.grid(1)
    for _ in range(100):
        if (i // n) % 2 == 0:
            y[i] = x[i] * 3
        else:
            y[i] = x[i] / 3
```

RES

LIDE



What is the purpose of pinned memory?

It ensure the OS doesn't swap out the data

0%

It places the memory on the GPU

0%

It prevents the memory from being changed

0%



51

Join at: vevox.app

ID: 104-345-753

Preparing Results



What is the purpose of pinned memory?

It ensure the OS doesn't swap out the data



65%

It places the memory on the GPU



8%

It prevents the memory from being changed



27%

RESULTS SLIDE

Back to the slides